

SymPy Bug Report

1 Bug 9: Finite Product Masks an Undefined Factor as Zero

1.1 Minimal Reproducer

```
from sympy import pi, product, sin, symbols

n = symbols("n")
term = sin(pi*n)/(n - 1)

print("term =", term)
print("product(term, (n, 0, 2)) =", product(term, (n, 0, 2)))
print("term values =", [term.subs(n, i) for i in [0, 1, 2]])
```

1.2 Observed Behavior

```
term = sin(pi*n)/(n - 1)
product(term, (n, 0, 2)) = 0
term values = [0, nan, 0]
```

On SymPy 1.14.0, the finite product is evaluated as 0, even though direct substitution of the individual factors shows that the middle factor is nan. The zero endpoint factors do not make a product containing an undefined factor well-defined.

1.3 Expected Behavior

The product should evaluate to nan, or otherwise remain unevaluated with the singular factor exposed. It should not collapse to 0.

1.4 Mathematical Explanation

The product requested from SymPy is

$$\prod_{n=0}^2 \frac{\sin(\pi n)}{n - 1}.$$

The three raw factors are

$$\frac{\sin(0)}{-1} = 0, \quad \frac{\sin(\pi)}{0} = \frac{0}{0}, \quad \frac{\sin(2\pi)}{1} = 0.$$

The middle factor is undefined. In SymPy's own arithmetic, direct multiplication of the substituted factors gives nan, not 0. Therefore the returned value is not an equivalent simplification, a harmless choice of representation, or a convention for an empty product; it loses the domain failure at $n = 1$.

1.5 Source-Level Diagnosis

The diagnosis locates the failure in `sympy/concrete/products.py`, inside `Product.doit`, with the relevant code around line 256. The public call `product` constructs a `Product` and calls `Product.doit`. At the start of `doit`, definite product variables are replaced by dummies with assumptions inherited from the concrete limits before the product evaluator runs. For the limits 0 and 2, the helper imported from `sympy/concrete/summations.py` creates an integer, nonnegative, real dummy.

```
for xab in self.limits:
    d = _dummy_with_inherited_properties_concrete(xab)
    if d:
        reps[xab[0]] = d
...
did = self.xreplace(reps).doit(**hints)
```

After replacing n by this assumption-enriched dummy, $\sin(\pi n)$ simplifies as zero for an integer index. Consequently, the whole product function becomes 0 before `_eval_product` sees the original denominator $n - 1$ and the singular point $n = 1$. The diagnosis notes that the downstream direct product evaluator in `sympy/concrete/products.py` would produce nan for the original raw factors, but it is bypassed because the pre-evaluated term is already zero. A plausible fix direction identified by the diagnosis is to check original finite-range factors for undefined values before accepting an identically zero product function, and more generally to avoid allowing strengthened index assumptions to mask denominator singularities in concrete products.

1.6 Additional Failing Instances

The independent verification covers a shifted family

$$\prod_{n=a-1}^{a+1} \frac{\sin(\pi n)}{n - a}, \quad a \in \{1, 2, 3, 4, 5, 6\}.$$

For every listed a , the factor at $n = a$ is 0/0, while the two adjacent integer factors have zero numerator. The expected result is therefore nan across the family, because every product contains an undefined factor.

Input / Expression	SymPy behavior	Expected behavior
$\prod_{n=0}^2 \sin(\pi n)/(n - 1)$	returns 0	nan
$\prod_{n=1}^3 \sin(\pi n)/(n - 2)$	returns 0	nan
$\prod_{n=2}^4 \sin(\pi n)/(n - 3)$	returns 0	nan
$\prod_{n=3}^5 \sin(\pi n)/(n - 4)$	returns 0	nan
$\prod_{n=4}^6 \sin(\pi n)/(n - 5)$	returns 0	nan
$\prod_{n=5}^7 \sin(\pi n)/(n - 6)$	returns 0	nan

1.7 Related Issues Assessment

The deduplication check identified only general concrete-product documentation as related background and did not find a public issue, pull request, Stack Overflow question, mailing-list thread,

or release-note entry for this exact finite product or for the specific failure where integer-index assumptions simplify $\sin(\pi n)$ to zero before the denominator singularity is checked. The verdict is a new failure mode with no public precedent. The bug remains individually actionable because the reproducer is minimal, the affected call is in SymPy's public concrete-product API, and the proposed regression test pins the exact finite-range behavior that should be preserved.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Mul, nan, pi, product, sin, symbols

@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
def test_finite_product_preserves_undefined_factor_before_zero_collapse(a):
    n = symbols("n")
    term = sin(pi*n)/(n - a)
    values = [term.subs(n, i) for i in range(a - 1, a + 2)]

    assert nan in values
    assert Mul(*values) is nan
    assert product(term, (n, a - 1, a + 1)) is nan
```